



Blazor Essencial

Blazor – Data Binding (Vinculação de dados)



@ Componentes - Data Binding

O **Databinding** no Blazor é o recurso que conecta a interface do usuário (UI) ao código do componente, permitindo que valores sejam exibidos (**one-way**) ou sincronizados automaticamente em ambas as direções (**two-way**).

1- One-Way DataBinding

É a vinculação onde os dados fluem apenas do código para a interface: **código → interface**. As mudanças no modelo aparecem na página, mas o que o usuário digita não altera o valor no código. Isso é feito usando símbolo **@** na marcação.

2- Two-Way DataBinding

Onde os dados fluem em ambas as direções: **código ⇔ interface**. Quando o usuário altera um valor na página, o código é atualizado; quando o código muda, a página reflete a mudança. Isso é feito usando **@bind**.

@ One-way DataBinding com @

@page “/one-way”

@rendermode InteractiveServer

<h3>Exemplo One-Way</h3>

<p style=“font-size: 40 px;”>Nome: @nome</p>

<button @onclick=“TrocarNome”>Trocar Nome</button>

```
@code {  
    private string nome = "Maria";  
  
    private void TrocarNome()  
    {  
        nome = "João"; // Ao clicar, a página é atualizada  
    }  
}
```

O valor **Maria** atribuído a **nome** aparece na página.(@nome)

Se o botão for clicado, o código altera o valor de nome para “João” que exibe na interface o novo nome e reflete a mudança.

O usuário não consegue digitar e mudar **nome** diretamente.

Os dados fluem apenas do código para a interface.

```
@page "/two-way"
```

```
@rendermode InteractiveServer
```

```
<h3>Exemplo Two-Way</h3>
```

```
<input @bind="nome" />
```

```
<p>Nome digitado: @nome</p>
```

```
@code {
```

```
    private string nome = "Maria";
```

```
}
```

Aqui o valor de **nome** é sincronizado:

- Se o usuário digitar no **input**, o valor da variável **nome** muda.
- Se o código mudar a variável, o input também reflete esta alteração

O binding padrão (**@bind**) usa o evento **onchange** do HTML, que só é disparado quando o elemento **perde o foco** (ao pressionar **TAB** ou clicar em outro elemento)

Os dados fluem em ambas as direções

@ Data Binding com @bind : usado com campo e propriedade

```
@page "/bind"
```

```
@rendermode InteractiveServer
```

```
<h1>Bind Exemplo com campo e propriedade</h1>
```

```
<p>
```

```
  <label> Valor: <input @bind="campo" /> </label>
```

```
</p>
```

```
<p>
```

```
  <label> Valor: <input @bind="Propriedade" /> </label>
```

```
</p>
```

```
<ul>
```

```
  <li><code>campo</code>: @campo</li>
```

```
  <li><code>propriedade</code>: @Propriedade</li>
```

```
</ul>
```

```
@code {
```

```
  private string? campo;
```

```
  private string? Propriedade { get; set; }
```

```
}
```

Quando usado com **propriedades**, o Blazor usa o **get para ler e o set para atribuir o valor**.

Quando usado em **campo**, a atribuição é direta **(sem get e set)**

No dia a dia, a prática recomendada é usar **propriedades**, porque dão mais flexibilidade *(podem ter lógica no **set ou get**)*.

```
private string? propriedade;  
private string? Propriedade  
{  
    get => propriedade;  
    set {  
        propriedade = value?.ToUpper();  
    }  
}
```

@ Diferença entre @bind e @bind:get e @bind:set

@bind: cria de forma automática e implícita um **get** e um **set** para a **propriedade**

@bind:get e **@bind:set**: permitem separar as duas responsabilidades (**ler e atualizar**), dando controle total sobre como o valor é exibido e modificado.

Diferente do **@bind** simples, eles também podem ser aplicados diretamente a **campos privados** ou **quando você não quer expor um set público**

@bind:get: define como obter o valor exibido no elemento da interface (UI).

@bind:set: define como processar e armazenar o valor quando o usuário altera o valor.

@bind:get e @bind:set

```
@page "/bind-get-set"
```

```
@rendermode InteractiveServer
```

```
h3>@bind:get e @bind:set</h3>
```

```
<input @bind:get="texto"
       @bind:set="AtualizarTexto" />
```

```
<p>Valor armazenado: @texto</p>
```

```
@code {
    private string texto = "";

    private void AtualizarTexto(string valor)
    {
        // Lógica personalizada antes de salvar
        texto = string.IsNullOrEmpty(valor)
            ? "Valor inválido"
            : valor.Trim().ToUpper();
    }
}
```

@bind:get="texto" → mostra o valor atual da variável que é um campo privado

@bind:set="AtualizarTexto" → é usado quando o usuário digitar algo, em vez de usar um set de uma propriedade, o Blazor vai chamar o método **AtualizarNome**. *(antes de armazenar valida)*

Use **@bind** em cenários simples de entrada de dados quando tiver uma propriedade com lógica no set

Use **@bind:get** e **@bind:set** se tiver um campo privado ou quiser separar leitura e escrita em métodos diferentes sem usar propriedade

@ Componentes : @bind:event="{EVENT}"

```
<input @bind="nome" />
```

Por padrão o valor da variável **nome** só é atualizado quando o **input perde o foco** e isso ocorre no evento **onchange**, mas podemos trocar o evento que dispara a atualização do valor usando **@bind:event="{EVENT}"**

Exemplo usando **@bind:event:**

```
<input @bind="nome" @bind:event="oninput" />
```

Agora o valor é atualizado a cada tecla digitada (evento **oninput**).

@ Componentes : @bind:event="{EVENT}"

```
@page "/bind-event"
```

```
@rendermode InteractiveServer
```

```
<h3>Exemplo com @bind:event</h3>
```

```
<p>Digite algo:</p>
```

```
<input @bind="texto" @bind:event="oninput" />
```

```
<p>Valor atual: @texto</p>
```

```
@code {  
    private string texto = "";  
}
```

Com **@bind** normal → só atualiza quando o usuário pressiona **Tab** ou clica fora do campo.

Com **@bind:event="oninput"** → o valor aparece atualizado em tempo real, enquanto o usuário digita.

@bind = usa **onchange** por padrão (atualiza quando sai do campo).

@bind:event="{EVENT}" = permite escolher outro evento, como **oninput**, **onclick**, etc.

O mais usado na prática é **oninput** para **inputs** que precisam de atualização em tempo real

@ Data Binding - @bind:after (.NET 8+)

```
@page "/bind-after"
```

```
@rendermode InteractiveServer
```

```
<h3>Exemplo de @bind:after</h3>
```

```
<p>  
  Digite algo:  
  <input @bind="Nome" @bind:after="AoAtualizarNome" />  
</p>
```

```
<p>Nome digitado: @Nome</p>
```

```
@code {  
  private string Nome { get; set; } = "";  
  
  private void AoAtualizarNome()  
  {  
    Console.WriteLine($"O valor do Nome foi atualizado para: {Nome}");  
  }  
}
```

Serve para executar código imediatamente após a atualização do valor de um **@bind**.

@oninput ou **@onchange** disparam antes de você ter certeza que o valor foi aplicado na propriedade, o **@bind:after** garante que você só execute algo depois que o valor já foi atualizado na propriedade.

Usuário digita no **<input>**.

O **@bind** atualiza automaticamente a propriedade **Nome**.

Depois da atualização, o Blazor chama o método definido em **@bind:after** (neste caso, **AoAtualizarNome**).

@ Data Binding – @bind:format

O **@bind** sozinho funciona bem com string, mas quando usamos outros tipos (**como DateTime, decimal, double**), ele mostra o valor sem formatação ou usa a conversão padrão do .NET, que nem sempre é o que queremos.

O **@bind:format** é usado quando você quer mostrar um valor formatado (**como moeda, datas, casas decimais**) e ainda permitir que o usuário edite, mantendo a propriedade sincronizada no tipo original.

O **@bind:format** **não altera o valor real da propriedade** no código C#; ele serve apenas para formatar a conversão entre o valor e a representação no **input HTML**.

O **@bind:format** é **apenas visual na hora de exibir no input**, o valor não fica formatado internamente; ele sempre é convertido para o tipo da propriedade.



Data Binding – @bind:format

```
@page "/bind-format-date"
```

```
@rendermode InteractiveServer
```

```
<h3>Exemplo de @bind:format com Data</h3>
```

```
<p>
```

```
    <label> Data de nascimento:
```

```
        <input type="text" @bind="dataNascimento" @bind:format="dd/MM/yyyy" />
```

```
    </label>
```

```
</p>
```

```
<p>
```

```
    Data no modelo: @dataNascimento //inlui a hora no formato : 00:00:00
```

```
</p>
```

```
@code {
```

```
    private DateTime dataNascimento = new DateTime(2025,8, 25);
```

```
}
```

A cultura atual da aplicação influencia a interpretação dos valores.

Ex: **pt-BR**, separador decimal é , → 100,50.

Em **en-US**, separador decimal é . → 100.50.

Se o usuário digitar um valor que não pode ser convertido, ocorre erro de binding (*pode ser tratado com validação*).

O input exibirá a data : 25/08/2025

Se o usuário alterar para 01/01/2000, o **DateTime** será atualizado corretamente

O **@bind:format** é apenas para a entrada de dados (**input**)